R3.05 · TD-TP 02

PROGRAMMATION SYSTÈME

Le langage C — partie 2

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

Une boutique de maquettes Star Wars vient d'ouvrir à Mos Eisley et son logiciel d'inventaire plante une fois sur deux. Le LAMAE t'envoie en mission : reconstruire le stock avec des tableaux et des structures… sans laisser un seul octet déborder.

OBJECTIFS DE LA SÉANCE

- ☐ Déclarer, initialiser et parcourir des tableaux à une et deux dimensions.
- ☐ Comprendre la conversion d'un tableau en pointeur et ses conséquences sur `sizeof`.
- ☐ Manipuler des chaînes de caractères : le `'\0'` final, `strlen`, `snprintf`.
- ☐ Définir des structures (`struct`, `typedef`), y accéder avec `.`, les copier, les ranger dans un tableau.
- ☐ Détecter les accès hors bornes avec `-fsanitize=address` et valider tout indice saisi.

FIL ROUGE **Opération « Inventaire de Mos Eisley »**

Chaque mission rapporte un **drapeau** `FLAG{...}` à noter dans `badge.txt`. La dernière mission assemble tout : le programme d'inventaire complet de la boutique (fichier de départ `code/sujet02/boutique_depart.c`). Les exercices vont de ★ (incontournable) à ★★★ (pour aller loin). Un·e agent·e du LAMAE ne lit **jamais** une case qui n'existe pas.

Mission	Contenu	Durée
1 · La soute	Tableaux à une dimension, fonctions somme, min, max	25 min
2 · Hors des cartes	Accès hors bornes, `sizeof` et passage en paramètre	15 min
3 · Les transmissions	Chaînes de caractères et `'\0'`	20 min
4 · La carte galactique	Tableaux à deux dimensions	15 min
5 · Les plans	Structures, `typedef`, alignement mémoire	15 min
6 · La boutique	Tableau de structures : inventaire, recherche, tri	30 min
Bonus	Pour les rapides : panier de commande sécurisé	—

• • • compilation recommandée en R3.05

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g prog.c -o prog
$ gcc -std=c17 -Wall -Wextra -pedantic -g -fsanitize=address,undefined prog.c -o prog   # mise au point
```

Mission 1 — La soute : les tableaux

Un **tableau** est un ensemble fini d'éléments **de même type**, stockés en mémoire à des adresses **contiguës**. On le déclare ainsi :

```c
type nomDuTableau[nombreElements];
```

où `nombreElements` est une expression constante entière strictement positive. Par exemple `int tab[10];` réserve 10 `int` consécutifs, soit `10 * sizeof(int)` octets (40 octets si un `int` fait 4 octets, ce qui est le cas sur nos machines mais n'est pas garanti par la norme).

Pour plus de clarté, on nomme la taille avec une directive du préprocesseur :

```c
#define NB_ELEMENTS 10   // macro remplacée par le préprocesseur avant la compilation
```

On accède à un élément avec l'opérateur `[]`. Les éléments sont **toujours** numérotés de `0` à `nombreElements - 1`.

```c
#include <stdio.h>
#define N 10

int main(void)
{
    int tab[N] = {0};             // tous les éléments à 0
    for (int i = 0; i < N; i++)   // i va de 0 à N-1 : jamais i <= N !
        printf("tab[%d] = %d\n", i, tab[i]);
    return 0;
}
```

Un tableau ne peut pas figurer à gauche d'une affectation : on ne peut pas écrire `tab1 = tab2;`. Il faut copier élément par élément (ou utiliser `memcpy` de `<string.h>`) :

```c
int tab1[N], tab2[N] = {1, 2, 3};
for (int i = 0; i < N; i++)
    tab1[i] = tab2[i];
```

> **REMARQUE** **Tableau ou pointeur ?**
>
> Un tableau **n'est pas** un pointeur. Mais dans presque toutes les expressions, le nom d'un tableau est **converti implicitement** en un pointeur vers son premier élément (`tab` vaut `&tab[0]`). Ce pointeur n'est pas modifiable, d'où l'impossibilité d'écrire `tab1 = tab2`. Exceptions notables à la conversion : `sizeof tab` et `&tab`. Nous détaillerons tout cela au sujet 03.

Initialisation

On peut initialiser un tableau lors de sa déclaration par une liste de constantes :

```c
#include <stdio.h>
#define N 4

int main(void)
{
    int tab[N] = {1, 2, 3, 4};
    for (int i = 0; i < N; i++)
        printf("tab[%d] = %d\n", i, tab[i]);
    return 0;
}
```

Si la liste contient **moins** de valeurs que la dimension du tableau, les éléments restants sont **mis à zéro**, que le tableau soit global, `static` ou local. En revanche, un tableau **local sans aucun initialiseur** contient des valeurs indéterminées (ce qui traînait en mémoire), alors qu'un tableau global ou `static` est toujours initialisé à zéro.

Un tableau de caractères peut être initialisé par une chaîne littérale. Le compilateur ajoute le caractère nul `'\0'` à la fin : le tableau doit donc avoir **au moins un élément de plus** que le nombre de caractères visibles.

```c
#define N 8
char mot[N] = "exemple";   // 7 lettres + '\0' = 8 octets
```

Si l'on omet la dimension, elle est déduite de l'initialisation. Le programme suivant affiche donc 8 :

```c
#include <stdio.h>

int main(void)
{
    char mot[] = "exemple";
    printf("Nombre de caractères du tableau = %zu\n", sizeof mot / sizeof mot[0]);
    return 0;
}
```

> **PIÈGE CLASSIQUE** **%zu pour un sizeof**
>
> `sizeof` renvoie un `size_t` (entier **non signé**, 8 octets sur nos machines). Le bon format est `%zu`, pas `%d` : `-Wall` vous le signale.

EX 1.1 Somme de la cargaison

★ 10 min

Écrivez une fonction `int somme(const int tab[], int n)` qui renvoie la somme des `n` éléments d'un tableau d'`int`. Testez-la dans `main` avec le tableau des masses (en tonnes) de la soute : `{12, 7, 30, 3, 18, 25, 9, 4}`.

Drapeau : `FLAG{` + la somme obtenue + `}`.

EX 1.2 Le plus léger, le plus lourd

★ 10 min

Écrivez deux fonctions : `int minimum(const int tab[], int n)` qui renvoie le plus petit élément, et `int maximum(const int tab[], int n)` qui renvoie le plus grand.

1. Avec quelle valeur initialisez-vous la variable qui mémorise le minimum ? Pourquoi `0` est-il un mauvais choix ?
2. Que doit faire votre fonction si `n` vaut 0 ? Proposez un comportement et documentez-le en commentaire.

3. Écrivez une fonction `int indiceDuMax(const int tab[], int n)` qui renvoie la **position** du maximum plutôt que sa valeur.

> **INDICE**
> Initialisez avec le **premier élément** du tableau, puis comparez à partir de l'indice 1. Le mot-clé `const` indique (et garantit) que la fonction ne modifie pas le tableau.

Drapeau : `FLAG{` + le minimum + `-` + le maximum + `}`.

> **POUR LES RAPIDES** **Moyenne et écart**
> Ajoutez une fonction `double moyenne(const int tab[], int n)`. Attention au piège de la mission 4 du sujet 01 : `somme / n` entre deux `int` est une division entière.

Mission 2 — Hors des cartes : les bornes d'un tableau

EX 2.1 La case fantôme ★★ 8 min

Récupérez `code/sujet02/hors_bornes.c` :

hors_bornes.c

```c
#include <stdio.h>
#define N 5

int main(void)
{
    int secret = 1977;
    int tab[N] = {10, 20, 30, 40, 50};
    for (int i = 0; i <= N; i++)        // erreur volontaire : <= au lieu de <
        printf("tab[%d] = %d\n", i, tab[i]);
    tab[N + 2] = 0;                     // écriture hors du tableau !
    printf("secret = %d\n", secret);
    return 0;
}
```

1. Compilez **sans** sanitizer et exécutez. Le programme plante-t-il ? Qu'affiche `tab[5]` ?
2. Recompilez avec `-fsanitize=address -g` et exécutez : quel est le nom de l'erreur signalée et à quelle ligne ?
3. Corrigez le programme.

> **SÉCURITÉ** **Le C ne vérifie pas les bornes**
> Lire `tab[N]` ou écrire `tab[N + 2]` est un **comportement indéfini** : le programme peut sembler fonctionner, afficher une valeur au hasard, modifier une autre variable (ici peut-être `secret`) ou planter. C'est exactement ce qu'exploitent les attaques par débordement de tampon. La règle : **tout indice venant de l'extérieur** (saisie, fichier, réseau) doit être vérifié avec `0 <= i && i < N` avant usage.

EX 2.2 Le sizeof qui ment ★★ 7 min

Récupérez `code/sujet02/taille_tableau.c`, compilez avec `-Wall -Wextra` et exécutez.

taille_tableau.c (extrait)

```c
void afficherTaille(int tab[])
{
    printf("dans la fonction : %zu éléments\n", sizeof tab / sizeof tab[0]);
}

int main(void)
{
    int tab[10] = {0};
    printf("dans main : %zu éléments\n", sizeof tab / sizeof tab[0]);
    afficherTaille(tab);
    return 0;
}
```

1. Pourquoi les deux affichages diffèrent-ils ? Que lit réellement le paramètre `int tab[]` ? Quel avertissement gcc affiche-t-il ?
2. Conclusion : comment une fonction peut-elle connaître la taille d'un tableau reçu en paramètre ?

Drapeau : `FLAG{` + le nombre affiché *dans la fonction* + `}`.

> **PIÈGE CLASSIQUE** **Passer la taille avec le tableau**
> Un tableau passé à une fonction est reçu comme un **pointeur** vers son premier élément : la taille est perdue. Passez **toujours** la taille en paramètre supplémentaire (`int tab[], int n`), comme le font `fgets`, `memcpy` ou `snprintf`.

Mission 3 — Les transmissions : chaînes de caractères

En C, une **chaîne de caractères** est simplement un tableau de `char` terminé par le caractère nul `'\0'` (code 0). Toutes les fonctions de `<string.h>` et le format `%s` s'appuient sur ce marqueur de fin : sans lui, elles lisent la mémoire jusqu'à rencontrer un octet nul… ou planter.

Fonction (`<string.h>`, `<stdio.h>`)	Rôle
`strlen(s)`	Nombre de caractères **avant** le `'\0'` (renvoie un `size_t`).
`strcmp(s1, s2)`	Compare deux chaînes : 0 si égales, négatif ou positif sinon.
`snprintf(dest, taille, format, ...)`	Écrit au plus `taille - 1` caractères **plus** le `'\0'`. Renvoie la longueur qu'aurait eue la chaîne complète.
`strcpy`, `strcat`, `sprintf`	**À éviter** : ne connaissent pas la taille de la destination.
`strncpy(dest, src, n)`	Piège : n'ajoute **pas** de `'\0'` si `src` est trop longue.

EX 3.1 Compter sans strlen

★ 8 min

Écrivez votre propre fonction `size_t longueur(const char s[])` qui compte les caractères jusqu'au `'\0'`. Comparez son résultat avec `strlen` et avec `sizeof` sur `char msg[32] = "Que la Force soit avec toi";`. Pourquoi `sizeof` et `strlen` diffèrent-ils ?

Drapeau : `FLAG{` + `strlen(msg)` + `}`.

EX 3.2 Message intercepté

★★ 12 min

L'Empire chiffre ses messages en décalant chaque **lettre majuscule** de 3 positions dans l'alphabet (chiffre de César : `A` devient `D`, `X` devient `A`).

1. Écrivez une fonction `void dechiffrer(char s[])` qui décale chaque majuscule de 3 positions **en arrière** et laisse les autres caractères inchangés.
2. Déchiffrez le message `"OD EDVH HVW VXU KRWK"`.
3. Construisez dans un tableau `char rapport[24]` la chaîne `"Base : <nom>"` avec `snprintf`. Que se passe-t-il si le nom est trop long ? Testez la valeur de retour de `snprintf` pour détecter la troncature et afficher un avertissement sur `stderr`.

INDICE

Un `char` est un entier : `'D' - 'A'` vaut 3. Pour un décalage circulaire : `'A' + (c - 'A' + 26 - 3) % 26`.

SÉCURITÉ **Toujours borner les copies de chaînes**

`strcpy(rapport, nom)` écrit autant d'octets que `nom` en contient, quelle que soit la taille de `rapport` : c'est un débordement de tampon garanti si `nom` vient de l'utilisateur. Le guide ANSSI recommande des fonctions qui prennent la taille de la destination en paramètre (`snprintf`) et la vérification systématique de la troncature.

Mission 4 — La carte galactique : tableaux à deux dimensions

On déclare un tableau à deux dimensions ainsi :

```c
type nomDuTableau[nombreLignes][nombreColonnes];
```

C'est en fait un tableau à une dimension dont chaque élément est lui-même un tableau : les lignes sont rangées **les unes à la suite des autres** en mémoire. On accède à un élément par `tab[i][j]`. Pour l'initialiser, on utilise une liste de listes :

```c
#include <stdio.h>
#define M 2
#define N 3

int main(void)
{
    int tab[M][N] = {{1, 2, 3}, {4, 5, 6}};
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            printf("tab[%d][%d] = %d\n", i, j, tab[i][j]);
    return 0;
}
```

EX 4 Repérer la flotte impériale

★★ 15 min

La carte d'un secteur est une grille de 8 lignes × 10 colonnes de caractères (`char carte[8][10]`). Les cases vides contiennent `'.'`, les vaisseaux impériaux `'E'`, les planètes `'O'`.

1. Initialisez la grille avec des `'.'` (deux boucles imbriquées), puis placez quelques `'E'` et `'O'` à la main.
2. Écrivez une fonction `void afficherCarte(char carte[8][10])` qui affiche la grille avec les numéros de ligne et de colonne.

3. Demandez à l'utilisateur des coordonnées de tir `ligne colonne`. **Vérifiez** le retour de `scanf` **et** les bornes avant d'accéder à la case. Affichez « Touché ! » et remplacez le `'E'` par `'X'`, ou « Dans le vide ». Le jeu continue jusqu'à ce que tous les vaisseaux soient touchés.
4. Affichez les adresses `&carte[0][0]`, `&carte[0][9]` et `&carte[1][0]` avec `%p` (en convertissant en `(void *)`). Que constatez-vous ?

Drapeau : `FLAG{` + l'écart en octets entre `&carte[1][0]` et `&carte[0][0]` + `}`.

> **SÉCURITÉ** **Coordonnées hostiles**
>
> Si le joueur tape `-1 3` ou `8 12`, un programme non vérifié écrit `'X'` **en dehors** de la carte. Testez ces valeurs avec `-fsanitize=address` avant et après avoir ajouté votre contrôle de bornes. Une saisie se valide toujours en deux temps : **le format** (retour de `scanf`) puis **la plage** des valeurs.

Mission 5 — Les plans : les structures

Une **structure** regroupe un nombre fini d'objets de **types différents**. Chaque élément, appelé **membre** ou **champ**, est désigné par un identificateur. On distingue la déclaration du *modèle* de structure de celle d'un *objet* de ce type :

```c
struct Modele {          // déclaration du modèle
    type1 membre1;
    type2 membre2;
    /* ... */
};

struct Modele objet;     // déclaration d'un objet de ce type
```

On accède aux membres avec l'opérateur `.` : `objet.membre1`. Les membres sont rangés en mémoire **dans l'ordre de leur déclaration**, mais le compilateur peut insérer des octets de **remplissage** (*padding*) entre eux pour respecter l'alignement des types. Exemple : la norme d'un nombre complexe.

```c
#include <stdio.h>
#include <math.h>        // sqrt : compiler avec -lm

struct Complexe {
    double reelle;
    double imaginaire;
};

int main(void)
{
    struct Complexe z = {2.0, 2.0};          // initialisation comme un tableau
    double norme = sqrt(z.reelle * z.reelle + z.imaginaire * z.imaginaire);
    printf("norme de (%f + i %f) = %f\n", z.reelle, z.imaginaire, norme);

    struct Complexe z2;
    z2 = z;                                  // l'affectation de structures est permise
    printf("z2 = %f + i %f\n", z2.reelle, z2.imaginaire);
    return 0;
}
```

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g complexe.c -o complexe -lm
```

Contrairement aux tableaux, on **peut** affecter une structure à une autre (`z2 = z;`) : tous les membres sont copiés, y compris les tableaux qu'elle contient. On peut aussi initialiser par nom de champ (C99) : `struct Complexe z = {.reelle = 2.0, .imaginaire = -1.0};`. Pour alléger l'écriture, on définit souvent un alias avec `typedef` : `typedef struct Complexe Complexe;`.

EX 5.1 Opérations sur les complexes ★ 7 min

Écrivez les fonctions `Complexe addition(Complexe a, Complexe b)` et `Complexe produit(Complexe a, Complexe b)`, puis calculez `(2 + 2i) × (1 − i)`.

Drapeau : `FLAG{` + la partie réelle du produit (entière) + `}`.

EX 5.2 Le poids caché des structures ★★ 8 min

Affichez avec `%zu` la taille de ces deux structures, ainsi que la position de chaque champ avec la macro `offsetof(type, champ)` de `<stddef.h>` :

```c
struct A { char c1; int n; char c2; };
struct B { int n; char c1; char c2; };
```

1. Pourquoi `sizeof(struct A)` n'est-il pas égal à `1 + 4 + 1` ? Où sont les octets de remplissage ?
2. Pourquoi `struct B`, qui contient les mêmes champs, est-elle plus petite ?

> **CULTURE Le remplissage, une fuite d'information**
> Les octets de remplissage ne sont pas initialisés quand on remplit les champs un par un. Si l'on écrit une structure entière dans un fichier ou sur le réseau (sujets 05 et 09), on y envoie aussi ces octets… qui peuvent contenir des restes de données sensibles. D'où l'habitude défensive : `struct A a; memset(&a, 0, sizeof a);` ou `struct A a = {0};` avant de remplir.

Mission 6 — La boutique : un tableau de structures

La boutique vend huit maquettes. Chaque objet est décrit par un **nom**, une **référence**, une **date** de sortie et un **prix**.

Objet	Référence	Sortie	Prix
Faucon Millenium	SW-41	2021	129,90 €
Tie Advanced X1 Prototype	SW-31	2021	34,90 €
X-Wing Black One	SW-30	2021	39,90 €
Imperial Star Destroyer	SW-34	2021	89,90 €
TB-TT	SW-32	2021	44,90 €
Navette impériale	SW-33	2021	49,90 €
Étoile de la mort	SW-37	2021	99,90 €
Chasseur stellaire de Boba Fett	SW-38	2022	54,90 €

EX 6.1 Les fiches produits

★ 10 min

1. Définissez une structure `Date` (jour, mois, année) et une structure `Objet` contenant : `char nom[40]`, `char ref[8]`, `Date sortie`, `double prix` et `int stock`. Utilisez `typedef`.
2. Déclarez une variable `Objet faucon` initialisée avec les données du Faucon Millenium, puis affichez-la avec une fonction `void afficherObjet(const Objet *o)` (le `->` s'écrit `o->prix` ; c'est un raccourci de `(*o).prix`, détaillé au sujet 03).
3. Déclarez le tableau `Objet catalogue[NB_OBJETS]` contenant les huit objets (initialisation dans `boutique_depart.c`) et affichez tout le catalogue sous forme de tableau aligné (`%-32s`, `%8.2f`).

> **PIÈGE CLASSIQUE** **Les accents décalent les colonnes**
>
> « Navette impériale » est mal alignée par `%-32s` : en UTF-8, `é` occupe **2 octets**, et `printf` compte des octets, pas des caractères. Ce n'est pas un bug de votre code ; de même, `strlen("é")` vaut 2.

EX 6.2 Inventaire

★★ 10 min

Ajoutez au programme :

1. `double valeurStock(const Objet cat[], int n)` : valeur totale du stock (prix × quantité).
2. `int chercherParRef(const Objet cat[], int n, const char ref[])` : renvoie l'indice de l'objet de référence `ref`, ou `-1` s'il n'existe pas (comparaison avec `strcmp`).
3. Un menu (`switch`) : **1** afficher le catalogue, **2** chercher une référence, **3** valeur du stock, **4** objet le plus cher, **0** quitter. La référence est saisie avec `fgets` dans un tableau de 8 caractères (retirez le `'\n'` final).

Drapeau : `FLAG{` + la référence de l'objet le plus cher + `}`.

EX 6.3 Tri par prix

★★ 10 min

Triez le catalogue par prix croissant avec un **tri par sélection** : pour chaque position `i`, cherchez l'objet le moins cher parmi `cat[i..n-1]` et échangez-le avec `cat[i]`. Remarquez qu'échanger deux structures se fait simplement par affectation, grâce à une variable temporaire de type `Objet`.

> **SÉCURITÉ** **Le choix d'un article par numéro**
>
> Ajoutez au menu « **5** — acheter l'article n° k ». Le numéro est saisi par le client : avant d'écrire `catalogue[k].stock--`, vérifiez que la saisie a réussi, que `0 <= k < NB_OBJETS` **et** que le stock est strictement positif. Testez avec `-1`, `8`, `999999`, `abc`. Que se passerait-il sans ces contrôles ?

> **POUR LES RAPIDES** **Le panier de commande ★★★**
>
> 1. Ajoutez un panier : tableau `int panier[NB_OBJETS]` des quantités commandées. Le client peut ajouter plusieurs articles ; affichez une facture alignée avec le total.
> 2. Le prix est stocké en `double` : affichez `0.1 + 0.2 == 0.3`. Pourquoi les vrais logiciels de caisse stockent-ils les montants en **centimes**, dans un entier ? Modifiez la structure en conséquence.
> 3. Que se passe-t-il si un client commande 3 000 000 000 d'exemplaires et que la quantité est un `int` ? Ajoutez un contrôle avant la multiplication `prix × quantité` (voir le « Pour les rapides » du sujet 01).

Bilan de la séance

À RETENIR

- Les indices d'un tableau vont de `0` à `N - 1` ; le C ne vérifie **jamais** les bornes : c'est à vous de le faire.
- Une fonction qui reçoit un tableau reçoit un pointeur : on lui passe **aussi** sa taille.
- Une chaîne de caractères se termine par `'\0'` ; prévoir un octet de plus. On copie avec `snprintf`, jamais `strcpy`/`gets`.
- `sizeof` renvoie un `size_t`, qui s'affiche avec `%zu`.
- Une structure se copie par affectation ; ses champs sont dans l'ordre, avec un éventuel remplissage.
- Toute saisie est validée deux fois : le format (retour de `scanf`/`fgets`) puis la plage de valeurs.

- ☐ J'ai mes drapeaux dans `badge.txt` (missions 1 à 6).
- ☐ Mes programmes compilent sans **aucun** avertissement et passent sous `-fsanitize=address` sans erreur.

Aller plus loin

Ressources

Ressource	Pour quoi faire
`man 3 string`, `man 3 strlen`, `man 3 snprintf`	Les fonctions de chaînes, leurs valeurs de retour et leurs pièges (section BUGS de `strcpy`).
`man 3 offsetof`, `man 3 memcpy`, `man 3 qsort`	Position d'un champ ; copie de blocs ; tri générique (vu au sujet 04).
cppreference — tableaux et structures	Règles exactes d'initialisation, de conversion et d'alignement.
Kernighan & Ritchie, *Le langage C*, chap. 5 et 6	Tableaux, chaînes et structures, par les auteurs du langage.
M. Kerrisk, *The Linux Programming Interface* (No Starch Press)	La référence de la programmation système Linux, pour toute la suite du module.
Guide ANSSI (dossier du CM)	Règles sur les tableaux, les chaînes (fonctions bornées) et l'initialisation des structures.
`valgrind ./prog`	Détecte les lectures de mémoire non initialisée (tableau local sans initialiseur).
`gdb` : `print tab`, `print catalogue[2]`, `ptype Objet`	Afficher un tableau ou une structure entière pendant le débogage.

Pour approfondir

CULTURE **Les tableaux de taille variable (VLA)**

Depuis C99, on peut écrire `int t[n];` avec `n` connu seulement à l'exécution. C11 les a rendus optionnels et le guide ANSSI les déconseille : la taille n'est pas contrôlée et un `n` trop grand (ou négatif) fait déborder la pile **sans aucun message d'erreur**. Préférez l'allocation dynamique (`malloc`, sujet 04), dont on peut tester l'échec.

CULTURE **Des fonctions de chaînes plus sûres**

`strlcpy`/`strlcat` (BSD, disponibles dans la glibc depuis la version 2.38) et les fonctions de l'annexe K de C11 (`strcpy_s`…, peu implémentées) tentent de corriger les défauts de `strcpy`. En C standard portable, `snprintf` reste la solution la plus simple pour copier ou concaténer en bornant la taille.

CULTURE **Membres de tableau flexibles**

Une structure peut se terminer par un tableau sans taille, `struct Message { size_t len; char data[]; };`, alloué ensuite avec `malloc(sizeof(struct Message) + n)`. C'est la forme classique des en-têtes de paquets réseau ou des messages de taille variable (sujets 07 et 09).